

Circuitos Sequenciais

Projeto de lógica Síncrona

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

- Visão geral de circuitos sequenciais
- Circuitos síncronos
- Exemplos
- Análise de *timming*
- Diretrizes de projeto



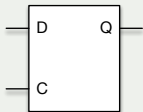
Visão geral

- Sequencial vs Combinacional:
 - Sequencial tem estado interno e
 - Saída é função das entradas e do estado.
- Elementos básicos de memória
 - Latch D
 - Flip-Flop D
 - Memória RAM
- Podem ser síncrono ou assíncrono.



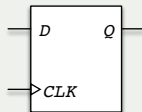
Latch D e Flip-Flop D (DFF)

■ Latch D:



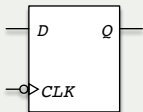
C	Q
0	Q_{prev}
1	D

■ DFF sensível à borda de subida:



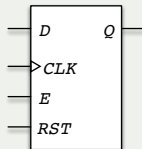
CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

■ DFF sensível à borda de descida:



CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

■ DFF com reset assíncrono e enable:



RST	E	CLK	Q
1	X	X	0
0	0	X	Q_{prev}
0	1	0	Q_{prev}
0	1	1	Q_{prev}
0	1		D



Diagrama de tempo simplificado

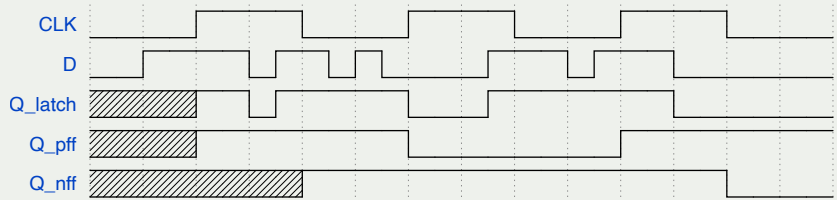
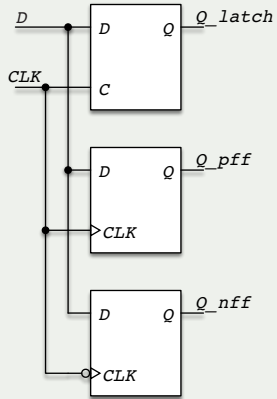
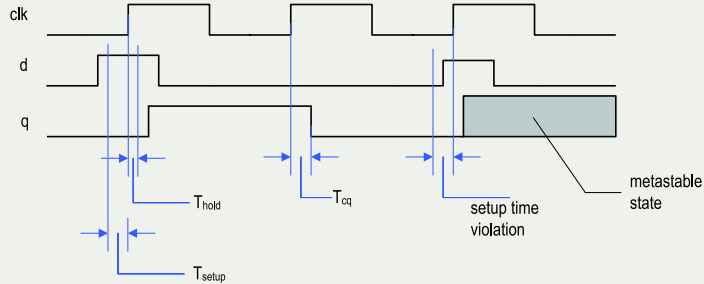


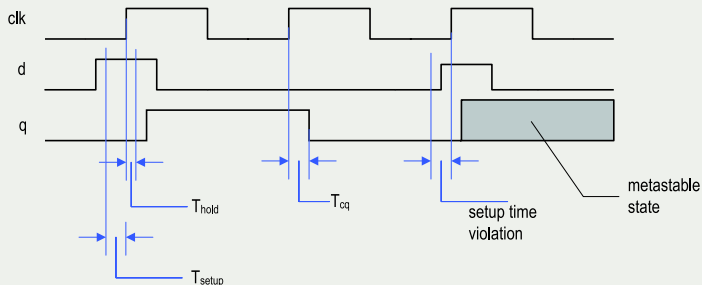
Diagrama de tempo detalhado - DFF



- T_{cq} : tempo do *clock-to-d*, o atraso de propagação necessário para que a entrada *d* apareça na saída *q* após a transição de amostragem do sinal de *clock*.



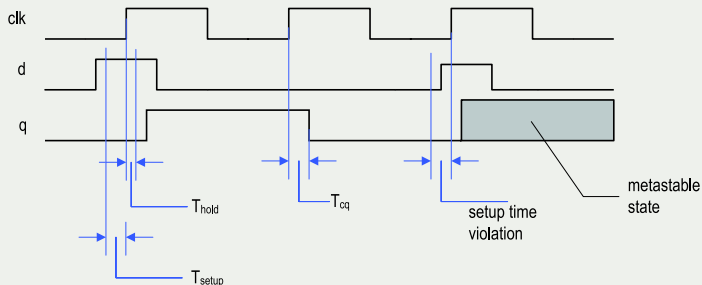
Diagrama de tempo detalhado - DFF



- T_{cq} : tempo do *clock-to-d*, o atraso de propagação necessário para que a entrada **d** apareça na saída **q** após a transição de amostragem do sinal de *clock*.
- T_{setup} : tempo de *setup* (configuração), o intervalo de tempo no qual o sinal **d** deve estar estável antes da transição do *clock*.



Diagrama de tempo detalhado - DFF



- T_{cq} : tempo do *clock-to-d*, o atraso de propagação necessário para que a entrada *d* apareça na saída *q* após a transição de amostragem do sinal de *clock*.
- T_{setup} : tempo de *setup* (configuração), o intervalo de tempo no qual o sinal *d* deve estar estável antes da transição do *clock*.
- T_{hold} : tempo de *hold* (aguarde), o intervalo de tempo no qual o sinal *d* deve permanecer estável depois da transição do *clock*.



Circuitos síncronos

- Circuito globalmente síncrono: todos os elementos de memória (D FFs) controlados (sincronizados) por um sinal de *clock* global comum.
- Circuito globalmente assíncrono, mas localmente síncrono (GALS).
- Circuito globalmente assíncrono:
 - Usa DFF, mas não um *clock* global.
 - Não usa sinal de *clock*.



- Um dos aspectos de projeto mais difíceis de um circuito sequencial:
 - Como satisfazer as restrições (*constraints*) de tempo (ex.: T_{setup} e T_{hold}).

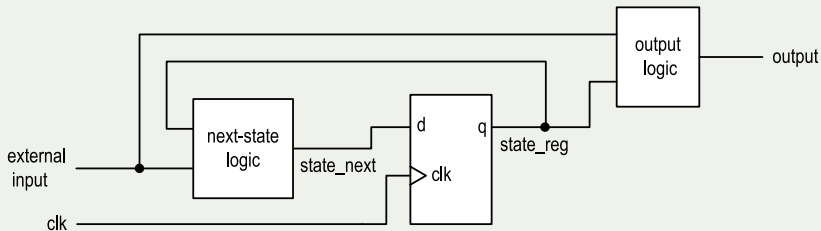


- Um dos aspectos de projeto mais difíceis de um circuito sequencial:
 - Como satisfazer as restrições (*constraints*) de tempo (ex.: T_{setup} e T_{hold}).
- A grande ideia: **MÉTODO SÍNCRONO**
 - Agrupe todos os DFFs em um único *clock*.
 - Só precisa lidar com a restrição de tempo de um elemento de memória.



■ Diagrama conceitual:

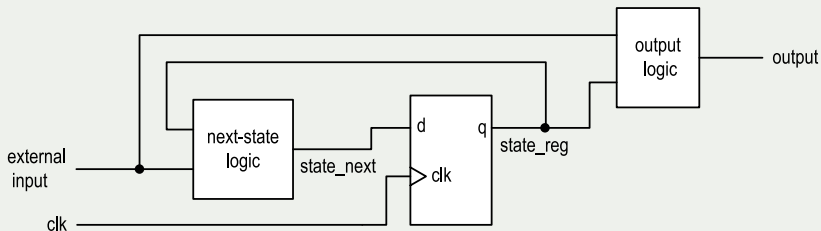
- Registrador de estado (elemento de memória)
- Lógica do próximo estado (circuito combinacional)
- Lógica de saída (circuito combinacional)



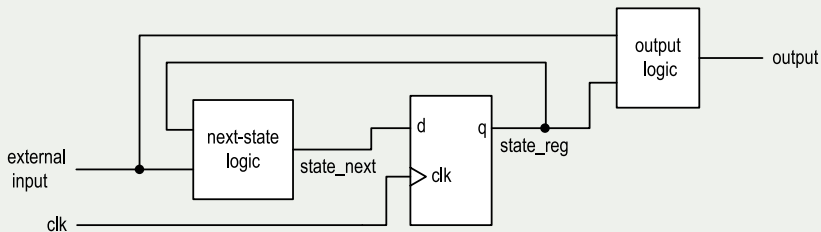
Circuito síncrono

■ Funcionamento:

- Na borda de subida do *clock*, o *state_next* é amostrado e armazenado no registrador, se tornando o novo valor de *state_reg* (permanece inalterado o resto do período do *clock*).
- Em paralelo:
 - A lógica do próximo estado determina o novo valor (novo *state_next*)
 - A lógica de saída gera os sinais de saída.
- Na próxima borda de subida do *clock*, o novo valor de *state_next* é amostrado e armazenado no registrador, repetindo todo o processo.



- *Glitches* não tem efeitos, desde que o `state_next` esteja estável na borda de subida do *clock*, ou seja, T_{setup} e T_{hold} sejam respeitados.

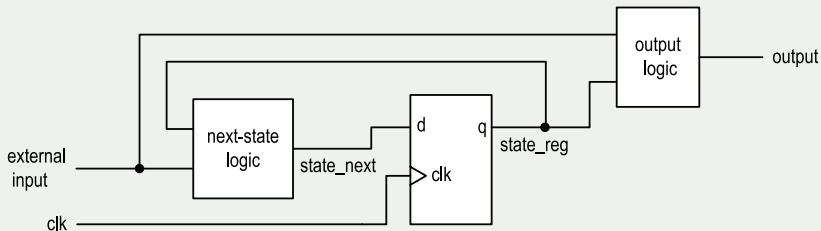


- Síntese: Como podemos separar os elementos de memória, o sistema é reduzido a um circuito combinacional. Todos os algoritmos e técnicas de otimização usados na síntese de circuitos combinados podem ser aplicados em conformidade.
- Análise de tempo (*timing*): Envolve apenas um ciclo de retorno. É simples, uma vez que o atraso de propagação do circuito de combinação é conhecido. Assim, a análise de tempo do circuito sequencial é essencialmente reduzida à análise de tempo de sua parte.
- Simulação: Simulação baseada em ciclo ignora o retardo de propagação exata, mas simula a operação do circuito de ciclo em ciclo. Como podemos identificar facilmente os elementos de memória e seu *clock*, a simulação baseada em ciclo pode ser usada para o projeto síncrono.



Circuito síncrono - Tipos

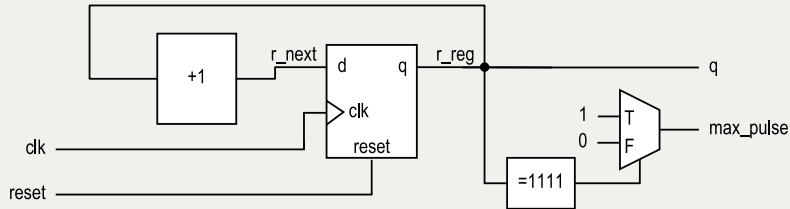
- Não definido formalmente, apenas para codificação.
- Três tipos:
 - Circuito sequencial “regular”.
 - Circuito sequencial “aleatório” (FSM)
 - Circuito sequencial “combinado” (FSM com um caminho de dados, FSMD)



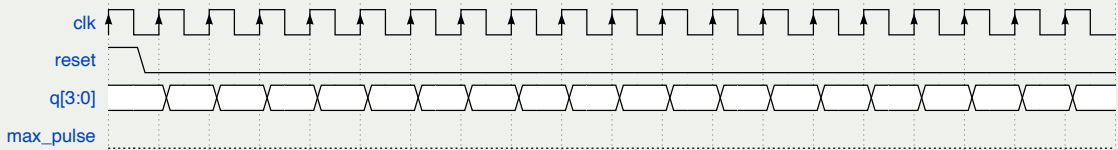
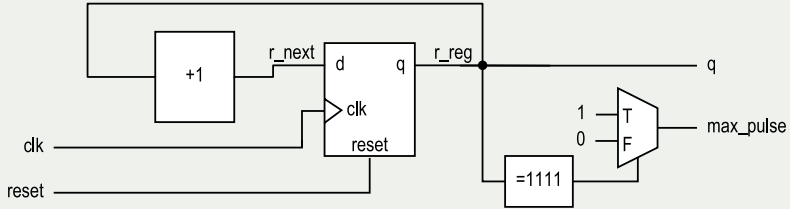
Exemplos

Exemplo 1: Contador binário livre com pulso de máximo

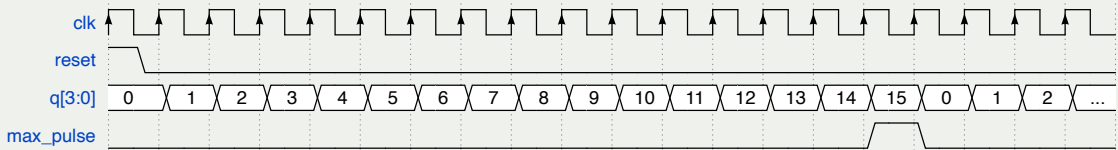
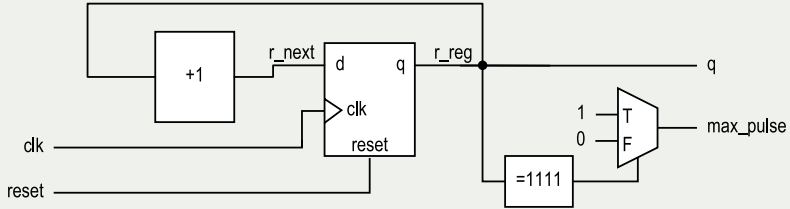
- Contagem em sequência binária
- Com uma saída `max_pulse`: pulso quando o contador está no estado “11 ... 11”
- Exemplo com 4 bits:



Exemplo 1: Contador binário livre com pulso de máximo

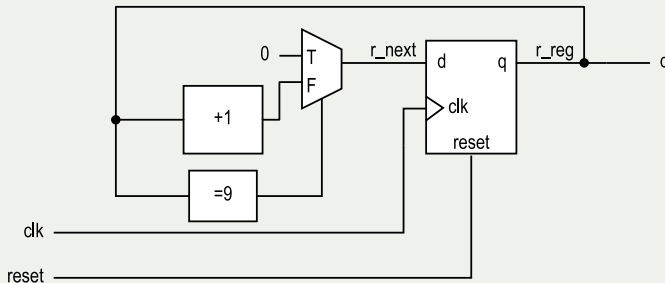


Exemplo 1: Contador binário livre com pulso de máximo

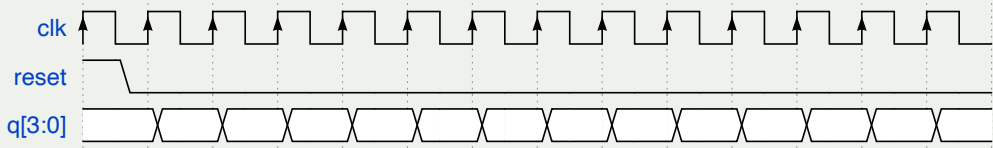
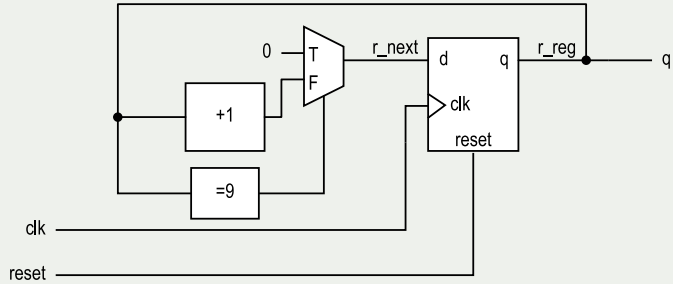


Exemplo 2: Contador decimal (*mod-10*)

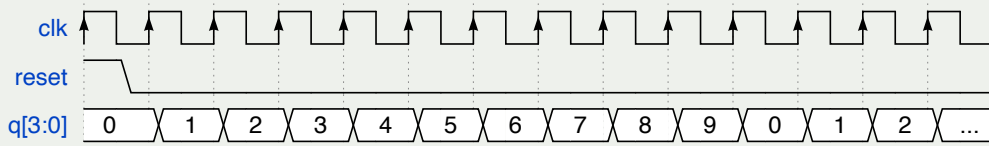
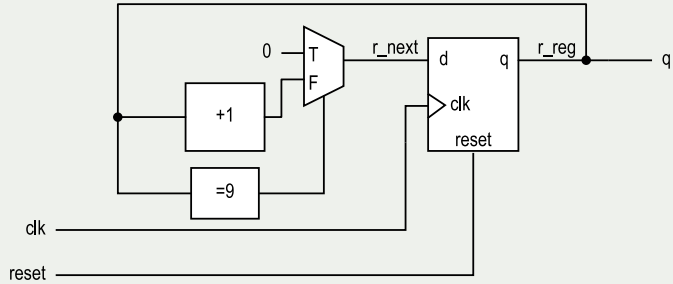
- Contador de 0 a 9.
- Como *mod-10* não é potência de 2, é necessário reiniciar a contagem após o 9.
- Necessário pelo menos 4 bits ($\lceil \log_2 10 \rceil$).



Exemplo 2: Contador decimal (*mod-10*)

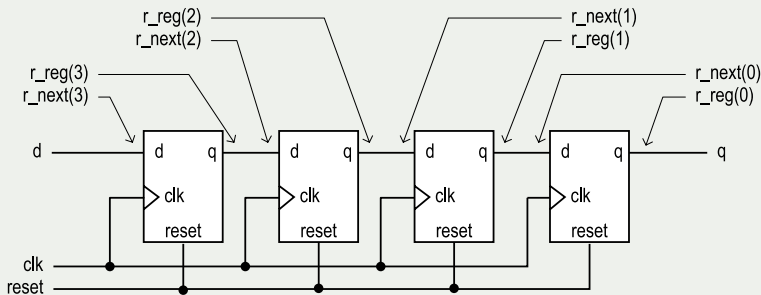


Exemplo 2: Contador decimal (*mod-10*)

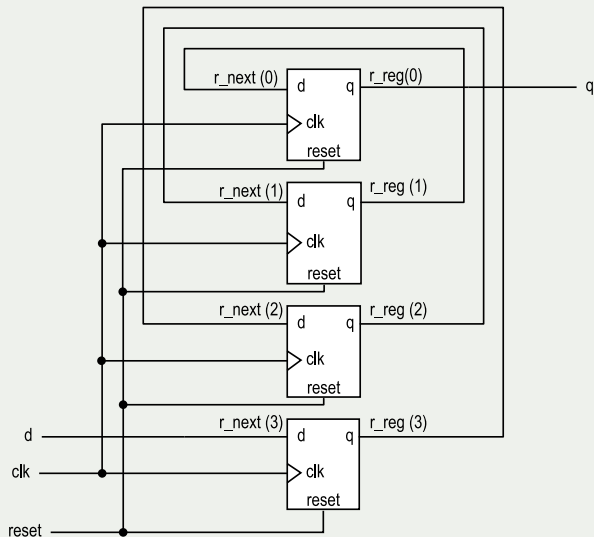


Exemplo 3: Registrador de deslocamento livre

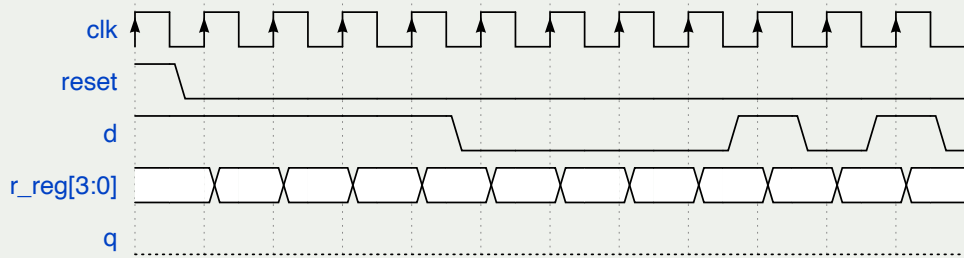
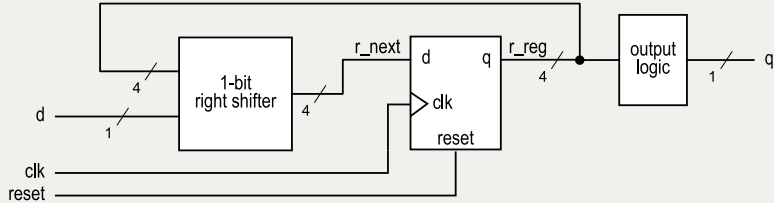
- Um registrador de deslocamento livre executa a operação de deslocamento continuamente.
- Não tem outros sinais de controle.



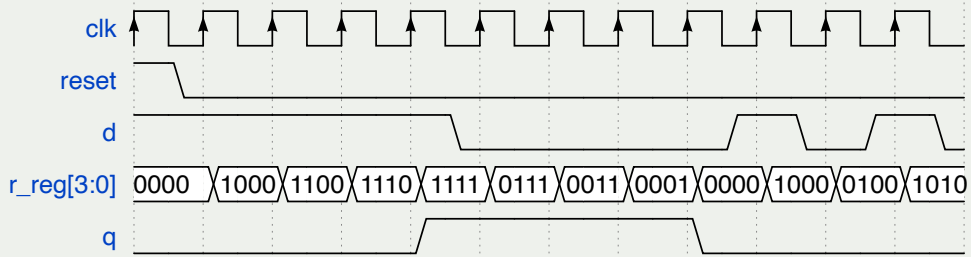
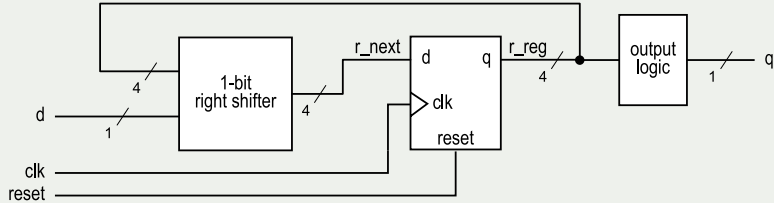
Exemplo 3: Registrador de deslocamento livre - forma vertical



Exemplo 3: Registrador de deslocamento livre - forma básica



Exemplo 3: Registrador de deslocamento livre - forma básica



Análise de *Timing*

- Circuito combinacional:
 - Caracterizado por atraso de propagação do caminho crítico.



- Circuito combinacional:
 - Caracterizado por atraso de propagação do caminho crítico.
- Circuito sequencial:
 - Deve satisfazer as restrições (*constraints*) de *setup* e *hold*.
 - Caracterizado pela taxa de *clock* máxima (por exemplo, contador de 200 MHz, processador de 2,4 GHz)
 - O tempo de *setup* (T_{setup}) e o atraso *clock-to-q* (T_{cq}) do registrador e o atraso de propagação da lógica do próximo estado estão embutidos na taxa de *clock*.

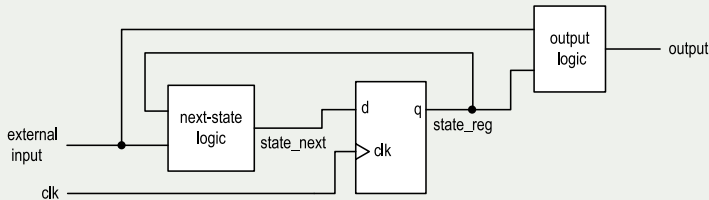


Implementação de célula padrão CMOS 55 μm

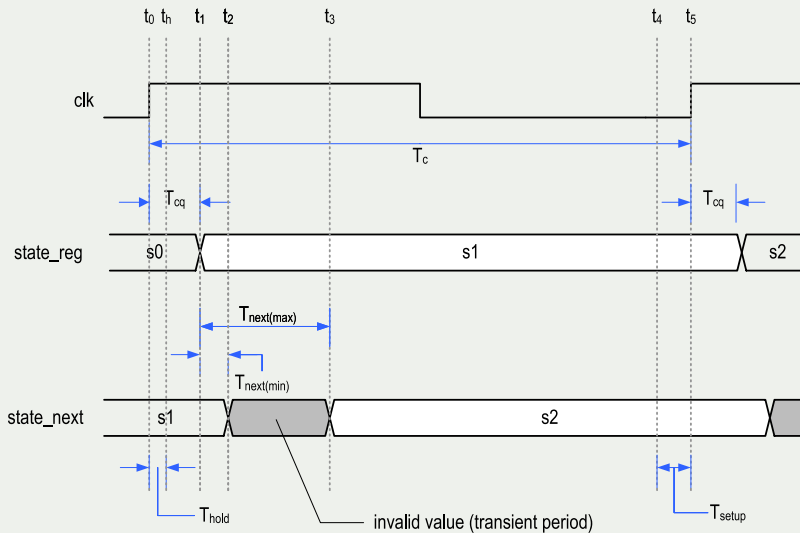
Width	VHDL operator									
	nand	xor	$>_a$	$>_d$	=	$+1_a$	$+1_d$	$+_a$	$+_d$	mux
Area (gate count)										
8	8	22	25	68	26	27	33	51	118	21
16	16	44	52	102	51	55	73	101	265	42
32	32	85	105	211	102	113	153	203	437	85
64	64	171	212	398	204	227	313	405	755	171
Delay (ns)										
8	0.1	0.4	4.0	1.9	1.0	2.4	1.5	4.2	3.2	0.3
16	0.1	0.4	8.6	3.7	1.7	5.5	3.3	8.2	5.5	0.3
32	0.1	0.4	17.6	6.7	1.8	11.6	7.5	16.2	11.1	0.3
64	0.1	0.4	35.7	14.3	2.2	24.0	15.7	32.2	22.9	0.3



- `state_next` deve satisfazer a restrição (*constraint*).
- Para isso, deve-se considerar os efeitos de:
 - `state_reg`: pode ser controlado.
 - Entrada externa (`external input`) síncrona.
 - Entrada externa (`external input`) assíncrona.
- Abordagem:
 - Os 2 primeiros: ajuste a taxa de *clock* para evitar violação.
 - Último: usar “circuito de sincronização” para resolver a violação.



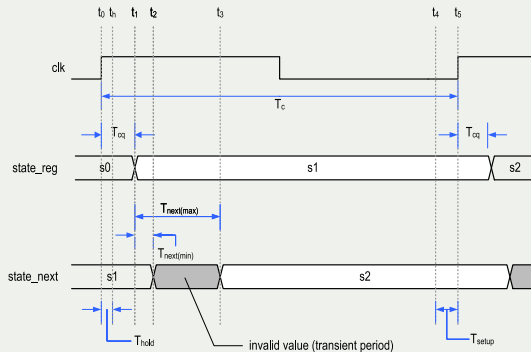
Análise de Timing



- O diagrama de tempo mostra o comportamento dos sinais `state_next` e `state_reg` durante um ciclo de *clock*.
 - t_0 marca a subida do *clock*, onde o sinal `state_next` se estabilizou e não mudará dentro dos períodos de *setup* e *hold*.
 - $t_1 = t_0 + T_{cq}$ e marca o momento em que a saída do registrador (`state_reg`) se estabilizou.
 - Uma vez que o `state_reg` é a entrada da lógica do próximo estado, a lógica do próximo estado responde de acordo.
 - $T_{next(min)}$ e $T_{next(max)}$ são os atrasos de propagação das respostas mais rápidas e mais lentas da lógica do próximo estado.
 - $t_2 = t_1 + T_{next(min)}$ marca o início da mudança do `state_next`.
 - $t_3 = t_1 + T_{next(max)}$ marca a estabilização do `state_next`.
 - t_5 é o fim do ciclo de *clock*, onde o `state_next` é amostrado e todo o processo se repete.



Violação do tempo de *setup* e taxa de *clock* máxima



- Para satisfazer o tempo de *setup*:

$$t_3 < t_4$$

- Sendo:

$$t_3 = t_0 + T_{cq} + T_{next(max)}$$

$$t_4 = t_5 - T_{setup} = t_0 + T_c - T_{setup}$$

- Reescrevendo:

$$t_0 + T_{cq} + T_{next(max)} < t_0 + T_c - T_{setup}$$

- Simplificando:

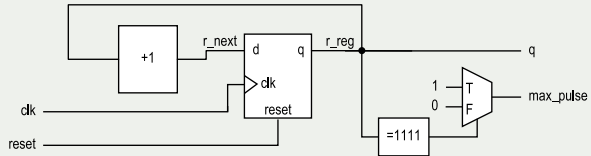
$$T_{cq} + T_{next(max)} + T_{setup} < T_c$$

- Temos:

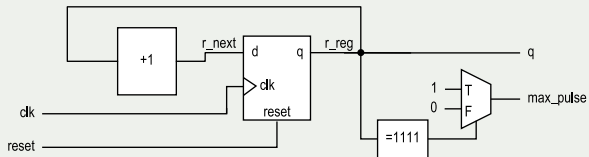
$$T_{c(min)} = T_{cq} + T_{next(max)} + T_{setup}$$



Exemplos de taxa de *clock*



Exemplos de taxa de *clock*



■ Exemplo de parâmetros do DFF:

- $T_{cq} = 1.0ns$

- $T_{setup} = 0.5ns$

■ Frequência máxima teórica:

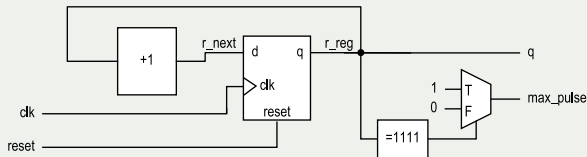
$$T_{c(min)} = T_{cq} + T_{next(max)} + T_{setup}$$

$$T_{c(min)} = T_{cq} + T_{setup} = 1.5ns$$

$$f_{max} = \frac{1}{T_{cq} + T_{setup}} \approx 666.7MHz$$



Exemplos de taxa de *clock*



Width	VHDL operator									
	nand	xor	> _a	> _d	=	+1 _a	+1 _d	+ _a	+ _d	mux
Area (gate count)										
8	8	22	25	68	26	27	33	51	118	21
16	16	44	52	102	51	55	73	101	265	42
32	32	85	105	211	102	113	153	203	437	85
64	64	171	212	398	204	227	313	405	755	171
Delay (ns)										
8	0.1	0.4	4.0	1.9	1.0	2.4	1.5	4.2	3.2	0.3
16	0.1	0.4	8.6	3.7	1.7	5.5	3.3	8.2	5.5	0.3
32	0.1	0.4	17.6	6.7	1.8	11.6	7.5	16.2	11.1	0.3
64	0.1	0.4	35.7	14.3	2.2	24.0	15.7	32.2	22.9	0.3

■ Exemplo de parâmetros do DFF:

■ $T_{cq} = 1.0ns$

■ $T_{setup} = 0.5ns$

■ Frequência máxima teórica:

$$T_{c(min)} = T_{cq} + T_{next(max)} + T_{setup}$$

$$T_{c(min)} = T_{cq} + T_{setup} = 1.5ns$$

$$f_{max} = \frac{1}{T_{cq} + T_{setup}} \approx 666.7MHz$$



Width	VHDL operator	
	+1 _a	+1 _d
Delay (ns)		
8	2.4	1.5
16	5.5	3.3
32	11.6	7.5
64	24.0	15.7

■ Otimizado em área para 8, 16 e 32 bits:

$$f_{max} = \frac{1}{T_{cq} + T_{8_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 2.4ns + 0.5ns} \approx 256.4MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{16_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 5.4ns + 0.5ns} \approx 142.9MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{32_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 11.6ns + 0.5ns} \approx 76.3MHz$$



Width	VHDL operator	
	+1 _a	+1 _d
Delay (ns)		
8	2.4	1.5
16	5.5	3.3
32	11.6	7.5
64	24.0	15.7

Width	VHDL operator	
	+1 _a	+1 _d
Delay (ns)		
8	2.4	1.5
16	5.5	3.3
32	11.6	7.5
64	24.0	15.7

■ Otimizado em área para 8, 16 e 32 bits:

$$f_{max} = \frac{1}{T_{cq} + T_{8_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 2.4ns + 0.5ns} \approx 256.4MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{16_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 5.4ns + 0.5ns} \approx 142.9MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{32_bit_inc(area)} + T_{setup}} = \frac{1}{1ns + 11.6ns + 0.5ns} \approx 76.3MHz$$

■ Otimizado em delay para 8, 16 e 32 bits:

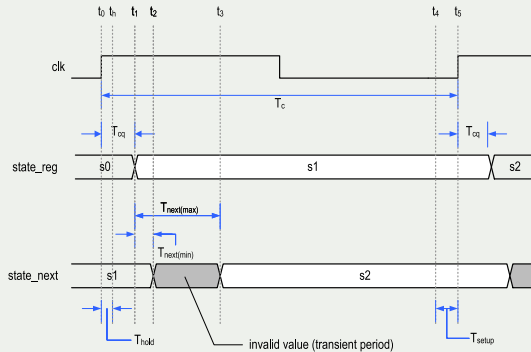
$$f_{max} = \frac{1}{T_{cq} + T_{8_bit_inc(delay)} + T_{setup}} = \frac{1}{1ns + 1.5ns + 0.5ns} \approx 333.3MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{16_bit_inc(delay)} + T_{setup}} = \frac{1}{1ns + 3.3ns + 0.5ns} \approx 208.3MHz$$

$$f_{max} = \frac{1}{T_{cq} + T_{32_bit_inc(delay)} + T_{setup}} = \frac{1}{1ns + 7.5ns + 0.5ns} \approx 111.1MHz$$



Violação do tempo de *hold*



- Para satisfazer o tempo de *hold*:

$$t_h < t_2$$

- Do diagrama:

$$t_2 = t_0 + T_{cq} + T_{next(min)}$$

$$t_h = t_0 + T_{hold}$$

- Reescrevendo:

$$t_0 + T_{hold} < t_0 + T_{cq} + T_{next(min)}$$

- Simplificando:

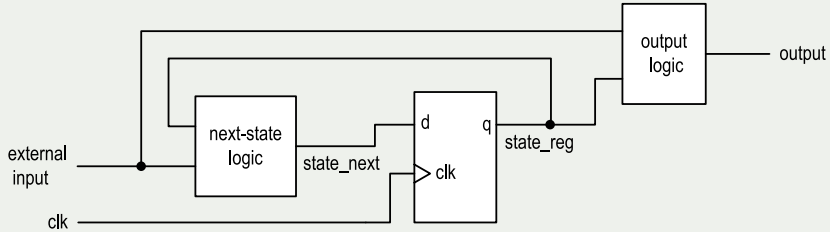
$$T_{hold} < T_{cq} + T_{next(min)}$$

- Temos:

$$T_{hold} < T_{cq}$$



Considerações de temporização relacionadas à saída



$$T_{co} = T_{cq} + T_{output}$$



Conclusão

- Seguir estritamente o método de projeto síncrono; ou seja, todos os registros em um sistema devem ser sincronizados por um sinal de *clock* global comum.
- O reset assíncrono, se usado, deve ser apenas para inicialização do sistema. Não deve ser usado para limpar os registros durante a operação normal.



Fim!